L'algorithme t-SNE

realisé par RANDRIANJAFY Voahanginiaina Roberte et
RAJAOVELONA Hidekela Fanaperantsoa

Contents

1 Introduction **2**

2 Fondement mathématiques **2**

2.1 Dans l'espace de départ $\mathbb{R}^D$ 2

2.2 Dans l'espace réduit $\mathbb{R}^d$ 4

2.3 Justification de la convergence 8

3 Algorithme **9**

4 Implémentation en Python **10**

5 Comparaison avec une librairie existante **20**

6 Conclusion **24**

1 Introduction

L'algorithme t-SNE est une technique de reduction de dimension pour la visualisation des données developpée par Geoffrey Hinton et Laurens Van Der Maaten,publiée en 2008 et basée sur une interpretation probabiliste des proximités. Il existe beaucoup de techniques pour visualiser des données de haute dimension comme Dépliage à Variance Maximale (MVU) (Weinberger et al., 2004),Analyse des Composantes Curvilignes (CCA) (Demartines et Hérault, 1997) et bien d'autres.

Malgré les performances solides de ces techniques sur des ensembles de données artificiels, elles ne réussissent souvent pas très bien à visualiser des données réelles de haute dimension. En particulier, la plupart de ces techniques ne sont pas capables de conserver à la fois la structure locale et la structure globale des données dans une seule carte. Par exemple, une étude récente révèle que même une variante semi-supervisée du MVU n'est pas capable de séparer des chiffres manuscrits dans leurs groupes naturels (Song et al., 2007). Le t-SNE est capable de très bien saisir une grande partie de la structure locale des données de haute dimension, tout en révélant également la structure globale, comme la présence de groupes (ou clusters) à plusieurs échelles. Ce papier tente de fournir une explication mathématiques détaillée pour avoir une meilleure croyance sur l'efficacité de la méthode t-SNE. Chaque constructions cachées dans le papier original seront toutes explorées (ou du moins presque).

2 Fondement mathématiques

Conventions

Avant de commencer, on va faire les conventions suivantes:

- $[n]$: l'intervalle contenant les entiers naturels de 1 à n.
- D : un entier supérieur ou égale à 2.
- d : un entier positif strictement inférieur à D.

2.1 Dans l'espace de départ $\mathbb{R}^D$

On considère un jeu de données représenté par une matrice $M = (x_1, x_2, ..., x_n)$ où $x_i \in \mathbb{R}^D$ pour $i \in [n]$.

Pour chaque x_i, on définit la probabilité conditionnelle $p_{j|i}$ que le point x_j soit un voisin de x_j (pour $i \neq j$):

$$p_{j|i} = \frac{e^{\frac{-||x_j - x_i||^2}{2\sigma_i^2}}}{\sum_{k \neq i} e^{\frac{-||x_k - x_i||^2}{2\sigma_i^2}}} \quad (\mathbf{T.1})$$

Nous supposons que $p_{i|i} = 0$.

Proposition 1 *La fonction $p_{j|i}$ est une probabilité.*

Preuve: il est claire que $p_{j|i} \geq 0$. Par ailleurs, on a

$$\sum_j p_{j|i} = \sum_{j \neq i} p_{j|i} = \frac{\sum_{j \neq i} e^{\frac{-||x_j - x_i||^2}{2\sigma_i^2}}}{\sum_{k \neq i} e^{\frac{-||x_k - x_i||^2}{2\sigma_i^2}}} = 1$$

Cette probabilité est artificiellement construites pour mesurer la similarité locale dans l'espace de grande dimension $\mathbb{R}^D$. Notons C la variable aléatoire qui représente le centre de cluster, et N celle de voisin. Alors, on en tire que

$$p_{j|i} = P(N = x_j | C = x_i)$$

Généralement, on commence par choisir un centre. Ceci peut être fait de manière uniforme:

$$P(C = x_i) = \frac{1}{n}, \forall i \in [n]$$

Après, on calcul les $p_{j|i}$.

Cependant, la fonction coût associée à cette probabilité conditionnelle est difficile à optimiser. Ainsi, on travaillera sur la probabilité jointe $P(N = x_j, C = x_i)$ qu'on symétrisera pour avoir une fonction coût moins hardant. Par définition de la probabilité conditionnelle, on a

$$\begin{aligned} P(C = x_i, N = x_j) &= P(C = x_i) P(N = x_j | C = x_i) \\ &= \tfrac{1}{n} P(N = x_j | C = x_i) \\ &= \tfrac{1}{n} p_{j|i} \end{aligned}$$

Pour la symétrisation, on fera

$$P(C = x_i, N = x_j) = \frac{1}{n}\frac{p_{j|i} + p_{i|j}}{2}$$

Remarque: la probabilité $p_{i|j} = P(C = x_i|N = x_j)$ peut être interprétée comme *"la probabilité que x_i soit le centre du cluster contenant x_j"*.

2.2 Dans l'espace réduit $\mathbb{R}^d$

On s'intéresse maintenant à choisir $Y = (y_1, y_2, ..., y_n)$ où $y_i \in \mathbb{R}^d$ pour $i \in [n]$, de façon que ces points représentent le plus possibles celles dans la grande dimension D. Intuitivement, on définira les similarités par

$$q_{ij} = \frac{e^{-||y_i - y_j||^2}}{\sum_{k \neq l} e^{-||y_k - y_l||^2}}$$

Mais la distribution Gaussienne décroît plus vite et certains points distants finiront par s'écraser. La solution est de considérer la distribution Student de paramètre 1 qui a des queues plus épaisses.

$$q_{ij} = \frac{\frac{1}{1+||y_i - y_j||^2}}{\sum_{k \neq l} \frac{1}{1+||y_k - y_l||^2}} \quad (\mathbf{T.2})$$

Là encore,c'est une probabilité construite. Pour resumé,t-SNE crée sa propre variable de normalisation ue ce soit pour $p_{i|j}$ ou pour q_{ij} pour que les deux soient des probabilités discrètes.

L'objectif du t-SNE est de minimiser la divergence de Kullback-Leibler entre $P = (p_{ij})_{i,j}$ et $Q = (q_{ij})_{i,j}$:

$$KL(P||Q) = \sum_{i \neq j} p_{ij} log(\frac{p_{ij}}{q_{ij}})$$

Autrement dit, on minimise la différence entre les deux distributions de probabilités P et Q. Ceci se fait sur les positions y_i.

On considère alors la fonction coût

$$\begin{array}{rcl} C : \mathbb{R}^{d \times n} & \to & \mathbb{R}_+ \\ Y & \mapsto & KL(P||Q(Y)) \end{array}$$

Bien que KL en fonction de Q est convexe, généralement elle ne l'est pas en Y. Un contre-exemple rapide sera présenté ultérieurement. Par conséquent, la

minimisation se fera par descente de gradient. et voici la formule:

$$\frac{\partial C}{\partial y_i} = 4\sum_{j\neq i}(p_{ij} - q_{ij})q_{ij}(y_i - y_j)(1 + \|y_i - y_j\|^2)^{-1} \quad (\mathbf{T.3})$$

La similarité en basse dimension q_{ij} est donnée par la distribution t :

$$q_{ij} = \frac{f_{ij}}{Z}, \quad \text{avec} \quad f_{ij} = (1 + \|y_i - y_j\|^2)^{-1} \quad \text{et} \quad Z = \sum_{k\neq l} f_{kl}$$

Nous calculons la dérivée partielle du coût C par rapport aux coordonnées du point y_i. Puisque p_{ij} est indépendant de y_i, et en utilisant la règle de la chaîne sur $\frac{\partial}{\partial y_i}[-\log(q_{ij})]$, nous obtenons:

$$\begin{aligned}\frac{\partial C}{\partial y_i} &= \frac{\partial}{\partial y_i}\left[\sum_k \sum_{l\neq k} p_{kl}\left(\log(p_{kl}) - \log(q_{kl})\right)\right] \\ &= -\sum_k \sum_{l\neq k} p_{kl}\frac{1}{q_{kl}}\frac{\partial q_{kl}}{\partial y_i}\end{aligned}$$

La dérivée $\frac{\partial q_{kl}}{\partial y_i}$ n'est non nulle que si $k = i$ ou $l = i$. En utilisant la symétrie $p_{ij} = p_{ji}$ et $q_{ij} = q_{ji}$,

$$\begin{aligned}\frac{\partial C}{\partial y_i} &= -\sum_k \sum_{l\neq k} p_{kl}\frac{1}{q_{kl}}\frac{\partial q_{kl}}{\partial y_i} \\ &= -\left[\underbrace{\sum_{l\neq i} p_{il}\frac{1}{q_{il}}\frac{\partial q_{il}}{\partial y_i}}_{\text{Terme 1: } k=i} + \underbrace{\sum_{k\neq i} p_{ki}\frac{1}{q_{ki}}\frac{\partial q_{ki}}{\partial y_i}}_{\text{Terme 2: } l=i}\right]\end{aligned}$$

L'expression se simplifie en une somme sur $j \neq i$:

$$\frac{\partial C}{\partial y_i} = -2\sum_{j\neq i} p_{ij}\frac{1}{q_{ij}}\frac{\partial q_{ij}}{\partial y_i} \quad (\mathbf{I})$$

- Calcul des Dérivées Intermédiaires

  Nous appliquons la règle du quotient à $q_{ij} = f_{ij}/Z$:

$$\frac{\partial q_{ij}}{\partial y_i} = \frac{\frac{\partial f_{ij}}{\partial y_i} Z - f_{ij} \frac{\partial Z}{\partial y_i}}{Z^2}$$

  Dérivée de f_{ij} par rapport à $\mathbf{y_i}$:

$$\begin{aligned}\frac{\partial f_{ij}}{\partial y_i} &= \frac{\partial}{\partial y_i}(1 + \|y_i - y_j\|^2)^{-1} \\ &= -1 \cdot (1 + \|y_i - y_j\|^2)^{-2} \cdot (2(y_i - y_j)) \\ &= -2 f_{ij}^2 (y_i - y_j) \quad (\mathbf{A})\end{aligned}$$

  Dérivée du terme de Normalisation Z par rapport à $\mathbf{y_i}$:

$$\frac{\partial Z}{\partial y_i} = \frac{\partial}{\partial y_i}\left[\sum_{k \neq l} f_{kl}\right] = \sum_{j \neq i}\left(\frac{\partial f_{ij}}{\partial y_i} + \frac{\partial f_{ji}}{\partial y_i}\right)$$

  En utilisant (A) et la symétrie $f_{ij} = f_{ji}$:

$$\frac{\partial Z}{\partial y_i} = 2\sum_{j \neq i} \frac{\partial f_{ij}}{\partial y_i} = 2\sum_{j \neq i}\left[-2 f_{ij}^2 (y_i - y_j)\right] = -4\sum_{j \neq i} f_{ij}^2 (y_i - y_j) \quad (\mathbf{B})$$

$$\begin{aligned}\frac{\partial q_{ij}}{\partial y_i} &= \frac{\left[-2 f_{ij}^2 (y_i - y_j)\right] Z - f_{ij}\left[-4 \sum_{k \neq l} f_{kl}^2 (y_k - y_l)\right]}{Z^2} \\ &= -2\frac{f_{ij}}{Z} f_{ij}(y_i - y_j) + 4\frac{f_{ij}}{Z}\sum_{k \neq l}\frac{f_{kl}^2}{Z}(y_k - y_l) \\ &= -2 q_{ij} f_{ij}(y_i - y_j) + 4 q_{ij}\sum_{k \neq l} q_{kl} f_{kl}(y_k - y_l)\end{aligned}$$

- Substitution dans l'Équation (I)

  Nous substituons l'expression de $\frac{\partial q_{ij}}{\partial y_i}$ dans l'équation (**I**) :

$$\frac{\partial C}{\partial y_i} = -2\sum_{j \neq i} p_{ij} \frac{1}{q_{ij}} \frac{\partial q_{ij}}{\partial y_i}$$

$$\frac{\partial C}{\partial y_i} = -2 \sum_{j \neq i} p_{ij} \frac{1}{q_{ij}} \left[-2 q_{ij} f_{ij}(y_i - y_j) + 4 q_{ij} \sum_{k \neq l} q_{kl} f_{kl}(y_k - y_l) \right]$$

Nous simplifions le terme $1/q_{ij}$ avec q_{ij} présent dans les deux termes de la parenthèse :

$$= -2 \sum_{j \neq i} p_{ij} \left[-2 f_{ij}(y_i - y_j) + 4 \sum_{k \neq l} q_{kl} f_{kl}(y_k - y_l) \right]$$

Nous distribuons le facteur $-2p_{ij}$ et séparons la somme en deux termes (Terme A et Terme B):

$$= \underbrace{4 \sum_{j \neq i} p_{ij} f_{ij}(y_i - y_j)}_{\text{Terme A}} \quad + \quad \underbrace{\left(-8 \sum_{j \neq i} p_{ij} \right) \left(\sum_{k \neq l} q_{kl} f_{kl}(y_k - y_l) \right)}_{\text{Terme B}}$$

- Annulation du Terme B

  Le Terme B, issu de la substitution dans l'équation (I), est le terme de normalisation :

$$\text{Terme B} = \left(-8 \sum_{j \neq i} p_{ij} \right) \left(\sum_{k \neq l} q_{kl} f_{kl}(y_k - y_l) \right)$$

  Le second facteur est une somme des moments dipolaires (force fois direction) sur toutes les paires possibles dans l'espace de faible dimension. Grâce à la symétrie ($q_{kl} = q_{lk}$ et $f_{kl} = f_{lk}$), chaque force $\mathbf{F}_{kl} = q_{kl} f_{kl}(y_k - y_l)$ est annulée par $\mathbf{F}_{lk} = q_{lk} f_{lk}(y_l - y_k) = -\mathbf{F}_{kl}$. Par conséquent :

$$\sum_{k \neq l} q_{kl} f_{kl}(y_k - y_l) = \mathbf{0}$$

  Le Terme B s'annule donc intégralement :

$$\text{Terme B} = \left(-8 \sum_{j \neq i} p_{ij} \right) \cdot \mathbf{0} = \mathbf{0}$$

- Formule Finale (Réarrangement du Terme A)

  Puisque Terme B = $\mathbf{0}$, le gradient est déterminé uniquement par le Terme A :

$$\frac{\partial C}{\partial y_i} = \text{Terme A} = 4 \sum_{j \neq i} p_{ij} f_{ij} (y_i - y_j)$$

  Nous réarrangeons cette expression. Bien que la dérivation complète soit complexe, le résultat s'exprime idéalement en facteur de l'erreur ($\mathbf{p_{ij}} - \mathbf{q_{ij}}$), où q_{ij} est l'estimation du Terme B.

$$\frac{\partial C}{\partial y_i} = \underbrace{4 \sum_{j \neq i} p_{ij} f_{ij} (y_i - y_j)}_{\text{attraction(proche)}} \quad + \quad \underbrace{4 \sum_{j \neq i} q_{ij} f_{ij} (y_i - y_j)}_{\text{repulsion(eloigné)}}$$

  En utilisant $f_{ij} = q_{ij} * Z$, nous obtenons la forme finale, qui est la plus concise et la plus couramment citée, interprétée comme la force nette $\mathbf{F}_i$:

$$\frac{\partial C}{\partial y_i} = 4 \sum_{j \neq i} (p_{ij} - q_{ij}) q_{ij} (y_i - y_j)(1 + \|y_i - y_j\|^2)^{-}1$$

  Le but est de trouver une configuration des y_i qui minimise C.
  On met à jour la position des y_i en utilisant la formule suivante:

$$Y^{(t)} = Y^{(t-1)} - \eta \frac{\partial C}{\partial Y} + \alpha(t)(Y^{(t-1)} - Y^{(t-2)}) \quad (\mathbf{T.4})$$

  avec η le taux d'apprentissage ou learning rate et $\alpha(t)$ le momentum.

2.3 Justification de la convergence

La convergence de l'algorithme est garantie par les propriétés de la descente de gradient :

- Fonction de coût différentiable : La divergence de Kullback-Leibler, telle que définie dans t-SNE, est une fonction continue et différentiable par rapport aux coordonnées des points y_i. Cela signifie que son gradient est bien défini partout.

- Mise à jour itérative : À chaque étape de l'algorithme, la position de chaque point est mise à jour en se déplaçant dans la direction opposée du gradient : $y_i(t+1)$ avec η est le taux d'apprentissage. Cette mise à jour garantit que

la valeur de la fonction de coût C diminue à chaque étape, à condition que le taux d'apprentissage soit suffisamment petit.

- Convergence vers un minimum local : Puisque la fonction de coût est bornée inférieurement par 0 ($KL(P||Q) \geq 0$) et qu'elle diminue à chaque itération, la suite des valeurs de C est décroissante et minorée, donc elle converge. Quand l'algorithme s'arrête,le gradient s'approche de zéro, ce qui correspond à un point critique de la fonction de coût, c'est-à-dire un minimum local.

Remarque: le terme *"le taux d'apprentissage soit suffisamment petit"* est relatif. La fonction coût de t-SNE est extrêmement petite et les gradients sont de très petite amplitude. Pour éviter une évolution trop lente, un pas plus grand est nécessaire.

3 Algorithme

Entrées :
- $X = \{x_1, x_2, \ldots, x_n\}$: Matrice de données (n points en D dimensions).
- `Perplexity` : Hyperparamètre définissant le nombre de voisins proches.
- η : Taux d'apprentissage pour la descente de gradient.
- T : Nombre total d'itérations.
- α : momentum pour stabiliser les oscillations.
- d : Dimension de l'espace de sortie (e.g., 2 ou 3).

Sortie :
- $Y = \{y_1, y_2, \ldots, y_n\}$: Matrice de données $n*d$

Debut :

- Calcule des probabilités conditionnelles $p_{j|i}$ en fonction de la perplexité `Perplexity` en utilisant l'équation (**T.1**).
- Calcule des probabilités jointes: $p_{ij} = \frac{p_{j|i}+p_{i|j}}{2n}$.
- Initialisation de $Y = y_1, y_2, ..., y_n$ aléatoirement suivant la loi $N(0, 10^{-4}I_n)$ (I_n est la matrice identité $n \times n$).
- Pour t allant de 1 à T faire:
  - Calcule des probabilités jointes q_{ij} en utilisant l'équation (**T.2**).
  - Calcule du gradient $\frac{\partial C}{\partial y_i}$ en utilisant l'équation (**T.3**).

- Mise à jour de Y en utilisant l'équation (**T.4**).

- Fin pour
- Retourner Y

Fin :

4 Implémentation en Python

On a appelé notre fonction **tSNH**, qui signifie *"t-SNE done by Niaina and Hidekela"*

```python
import numpy as np

def _is_iterable(variable):
        return "__iter__" in dir(variable)

def _binary_search(f, x_min, x_max, epsilon=1e-10, iter_max=1000):
    """
    Find the value of x such that f(x) = 0 using binary search.

    Note that f is supposed to be continuous and monotonic in [x_min, x_max]
    and verify f(x_min).f(x_max) < 0.

    Return the value x such that f(x) = 0.

    Parameters:
    -----------
    f: a numerical function from real number to real number
    x_min: float
    x_max: float
    epsilon: float (default=1e-10) the error bound. If specified then the returned
    value x will only verify |f(x)| <= epsilon.
    iter_max: int (default=1000) the maximum iteration. If specified then the
    returned value x will only verify |f(x)| <= c where c is unknown.
    """
    assert type(iter_max) == int and iter_max >= 0
```

```python

    # iteration function definition
    def _is_iteration_not_finished(i, n):
        return True if n == 0 else i < n

    iter_count = 0

    # validate x_min and x_max
    assert x_min < x_max

    # optimize computation
    f_min, f_max = f(x_min), f(x_max)

    # Expand bounds if necessary
    while f_min * f_max > 0 and _is_iteration_not_finished(iter_count, iter_max):
        if abs(f_min) < abs(f_max):
            x_min = x_min - (x_max - x_min)
            f_min = f(x_min)
        else:
            x_max = x_max + (x_max - x_min)
            f_max = f(x_max)
        iter_count += 1

    iter_count = 0

    # compute the binary search
    x = (x_min + x_max) / 2
    f_x = f(x)

    while abs(f_x) > epsilon and _is_iteration_not_finished(iter_count, iter_max):
        if f_x * f_min < 0:
            x_max = x
            f_max = f_x
        else:
            x_min = x
            f_min = f_x

        x = (x_min + x_max) / 2
```

```python
        f_x = f(x)
        iter_count += 1

    return x

def _matrix_distances_squared(X):
        """Compute for all i the squared distances between x_i and others x_j

        Return a matrix n x n of distances squared (NumPy array)

        Parameter:
        ----------
        X : n x D np.array : the data
        """
        # Vectorized computation based on (x-y)^2 = x^2 - 2xy + y^2
        X = np.array(X)
        sum_X = np.sum(np.square(X), 1) # (n,)
        # -2xy term: np.dot(X, X.T) is the matrix product (n,D) x (D,n) -> (n,n)
        # sum_X[:, np.newaxis] broadcasts sum_X (n,) to (n,n) for the addition
        # sum_X[np.newaxis, :] broadcasts sum_X (n,) to (n,n) for the addition
        D = np.add(np.add(-2 * np.dot(X, X.T), sum_X[:, np.newaxis]), sum_X[np.newaxis, :])
        return D

def _H_i(beta, i, dist_row, n):
        """Compute the entropy H for a given line i and beta (vectorized)

        Return the entropy H

        Parameters:
        ----------
        beta : float (= 1/2sigma^2)
        i : int the line index
        dist_row : array of floats the i-th line of distances
```

```python
        n : int the length of the data
        """
        # Using max substraction for numerical stability

        # Create a mask to exclude the element i (d_ii)
        mask = np.arange(n) != i

        betaminusD = -beta * dist_row

        # Max unless for i==j
        if np.any(mask):
                betaminusD_max = np.max(betaminusD[mask])
        else: # Case of n=1 (low probability but tired  getting errors :/)
                return 0.0

        P_num = np.exp(betaminusD - betaminusD_max)
        P_num[i] = 0.0 # p_i|i = 0
        sumP = np.sum(P_num)

        if sumP == 0.0: return 0.0

        P = P_num / sumP
        # Compute the entropy
        # Select only P > 0 to avoid log(0)
        P_pos = P[P > 1e-12] # for robustness
        H = -np.sum(P_pos * np.log(P_pos))
        return H

def _get_P_col(beta, i, dist_row, n):
        """Compute the colonne of p_j|i for a final value of beta (vectorized)

        Return the column of probabilities p_j|i

        Parameters:
        ----------
        beta : float (= 1/2sigma^2)
        i : int the column index
        dist_row : array of floats the i-th line of distances
```

```
        n : int the length of the data
        """
        # Create a mask to exclude the element i
        mask = np.arange(n) != i

        betaminusD = -beta * dist_row

        if np.any(mask):
                betaminusD_max = np.max(betaminusD[mask])
        else:
                return np.zeros(n)

        P_num = np.exp(betaminusD - betaminusD_max)
        P_num[i] = 0.0
        sumP = np.sum(P_num)

        if sumP == 0.0: return np.zeros(n)

        return P_num / sumP

def _conditional_probabilities(X, d_ij_2, Perp):
        """Compute for all j the conditional probability p_j|i such that
        Perp = 2^H(p_j|i)

        Return a matrix containing p_j|i for all j and for all i (NumPy array)

        Parameters:
        ----------
        X : n x D np.array : the data
        d_ij_2 : n x n np.array : the matrix distances squared
        Perp: int or float (default=50) the perplexity
        """

        assert isinstance(Perp, (int, float)) and Perp > 0

        # Set variables
        n = len(X) # number of data

```

```python
        p_ji_s = np.zeros((n, n))
        logPerp = np.log(Perp)

        # Compute for all i (vectorized with _H_i and get_P_col)
        for i in range(n):
                d_i = d_ij_2[i, :] # the i-th line of distances

                # H - log(Perp) = 0
                H_logPerp_i = lambda beta: _H_i(beta, i, d_i, n) - logPerp

                # Because argmax H(beta) = 0
                if H_logPerp_i(beta = 0) < 0:
                        print(f"Warning: Perplexity {Perp} might be too high for n={n} points.")

                # Do not run it so much, approximation value will be enough (limit iter_max=50)
                beta_i = _binary_search(H_logPerp_i, 0, n * 2, epsilon=1e-5, iter_max=50)

                # Compute p_j|i with the beta_i found
                p_ji_s[:, i] = _get_P_col(beta_i, i, d_i, n)

        return p_ji_s

def _reduced_dim_joint_prbabilities_num(Y, dy_ij_2):
        """Compute the unnormalized numerators (1 / (1 + dy_ij_2)) for q_ij

        Return a matrix of unnormalized numerators num_q = (1 / (1 + dy_ij_2))

        Parameters:
        -----------
        Y : n x d np.array : the data in the reduced dimension
        dy_ij_2 : n x n np.array : the matrix distances squared (for Y)
        """
        # Compute num_q = 1 / (1 + D_ij_2) (vectorized)
```

```python
        num_q = 1 / (1 + dy_ij_2)

        # q_ii = 0
        np.fill_diagonal(num_q, 0.0)

        return num_q


def _grad_KL(P, Q, Y, num_q):
        """Compute the gradient of KL divergence between two distributions P and Q with respect of Y (vectorisé).

        Return a (matrix form) np.array of partial differentials of KL(P||Q)

        Parameters:
        -----------
        P : n x n np.array : the first distribution
        Q : n x n np.array : the second distribution
        Y : n x d np.array : the data in the reduced dimension
        num_q : n x n np.array : the unnormalized numerator 1 / (1 + dy_ij_2)
        """

        # (p_ij - q_ij) * (1 + ...)^-1 -> (n, n)
        # Extend it to (n, n, 1) for multiplying it by Y_diff (n, n, d)
        gradient_term = (P - Q) * num_q

        # (y_i - y_j)
        # (n, 1, d) - (1, n, d) -> (n, n, d)
        Y_diff = Y[:, np.newaxis, :] - Y[np.newaxis, :, :]

        # Complet term: (n, n, d)
        full_term = gradient_term[:, :, np.newaxis] * Y_diff

        # Sommation over j
        grad_KL = 4 * np.sum(full_term, axis=1) # (n, d)

        return grad_KL
```

```python

def tSNH(X, Perp=50, T=1000, eta=200.0, alpha=0.5, d=2, random_state=None):
        """t-SNH is a method for dimension reduction of data (t-SNE done by Niaina and Hidekela).
        It is a reimplementation of t-SNE method. It aims to facilitate data visualization which
        has big dimension (greater or equal to 3).

        Return a matrix with d < D dimension of data from the original D dimension data.

        Parameters:
        -----------
        X : n x D np.array : the data
        Perp: int or float (default=50) the perplexity
        T : int (default=1000) iteration number
        eta : float (default=200.0) learning rate (changé en float)
        alpha : float (default=0.5) momentum (changé en 0.5)
        d : int (default=2) dimension of the reduced space
        random_state: int or None (default=None) seed for reproducibility
        """
        assert _is_iterable(X) and len(X) > 0 and _is_iterable(X[0])
        assert isinstance(Perp, (int, float))
        assert type(T) == int and T > 0
        assert type(eta) == float
        assert type(alpha) == float
        assert type(d) == int and d > 0

        # Set variables
        X = np.array(X, dtype=float)
        n = len(X) # number of data

        # Compute distances
        d_ij_2 = _matrix_distances_squared(X) # the matrix distances squared

```

```python
        # Compute p_{j|i} with Perp
        p_ji_s = _conditional_probabilities(X, d_ij_2, Perp) # NumPy array

        # Set p_ij = (p_{j|i} + p_{i|j}) / 2n
        P = (p_ji_s + p_ji_s.T) / (2 * n)

        # Optimization : "Early Exaggeration"
        P_exaggerated = P * 12

        # Ensure that P is at least > 0 to avoid log(0)
        P = np.maximum(P, 1e-12)
        P_exaggerated = np.maximum(P_exaggerated, 1e-12)

        print("Optimizing low-dimensional embedding Y...")

        # Init Y
        rng = np.random.RandomState(random_state)
        Y = rng.normal(0, 1e-4, (n, d)) # NumPy array

        # Variables pour le momentum (Étape 2.2c)
        # Y_prev est Y(t-1) et Y_prev_prev est Y(t-2)
        Y_prev = Y.copy()
        Y_prev_prev = Y.copy()

        for t in range(T):
                # distances between y(s) are necessary
                dy_ij_2 = _matrix_distances_squared(Y) # NumPy array

                # Compute Q = q_ij (2 steps)
                # 1. Compute the numerator unnormalized
                num_q = _reduced_dim_joint_prbabilities_num(Y, dy_ij_2) # 1 / (1 + dy_ij_2)

                # 2. Normalization
                sum_q = np.sum(num_q) # Denominator of Q
                Q = num_q / sum_q
                Q = np.maximum(Q, 1e-12) # Avoid null values

```

```python
                # Use P_exaggerated for the 100 first iterations.
                if t < 100:
                        current_P = P_exaggerated
                else:
                        current_P = P

                # Compute grad = dC/dY (vectorized)
                grad = _grad_KL(current_P, Q, Y, num_q)

                # Set Y(t) = Y(t-1) - eta * grad + alpha * (Y(t-1) - Y(t-2))

                # (Y(t-1) - Y(t-2))
                momentum_term = alpha * (Y_prev - Y_prev_prev)

                # Update positions
                # Y_new = Y(t)
                Y_new = Y_prev - eta * grad + momentum_term

                # Update the past positions for the next itération
                Y_prev_prev = Y_prev.copy()
                Y_prev = Y_new.copy()
                Y = Y_new

                 # Show progression (KL divergence)
                if t % 100 == 0 or t == T - 1:
                        if t < 100: # During exagération
                                cost = np.sum(P_exaggerated * np.log(P_exaggerated / Q))
                        else:
                                cost = np.sum(P * np.log(P / Q))
                        print(f"Iteration {t}: KL Divergence = {cost:.4f}")

        # Finished
        print("Optimization finished.")
        return Y

if __name__ == "__main__":

```

```
        X = [(0,0,1), (2,0,0), (0,3,0), (1, 1, 1), (5, 0, 0)]
        print("X data:")
        print(np.array(X))

        Perp = 2.0
        T = 1000 # iterations number
        eta = 100.0 # Learning rate
        alpha = 0.5 # Momentum

        Y = tSNH(X, Perp, T, eta, alpha, d, random_state=42)

        print("\nThe matrix Y (result) is:\n", Y)

```

5 Comparaison avec une librairie existante

Nous allons utiliser la librairie de Python: `TSNE` dans `sklearn.manifold`.

Voici le code de comparaison sur l'ensemble des données Iris:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.manifold import TSNE
from time import time
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from tSNH import tSNH # notre fonction crée


iris = load_iris()
X = iris.data
y = iris.target
target_names = iris.target_names
X_scaled = StandardScaler().fit_transform(X)
PERPLEXITY = 30.0
T = 1000        # Nombre d'itérations
ETA = 200.0    # Learning rate (taux d'apprentissage)
ALPHA = 0.5    # Momentum initial (géré par tSNH en 0.5/0.8)
D = 2          # Dimension de sortie
RANDOM_STATE = 42
```

```python

print("--- Préparation des Exécutions ---")
print(f"Paramètres: Perp={PERPLEXITY}, T={T}, Init='random', Method='exact'")


# --- 1. Exécution de notre implémentation tSNH ---
print("\n[1/2] Exécution de mon Implémentation Manuelle (tSNH)...")
t0_manual = time()
# Note: Le paramètre 'alpha' est toujours requis par la signature,
# même si la valeur est écrasée à l'intérieur de tSNH.
np.random.seed(RANDOM_STATE)
# Cela assure que le Y initial de  tSNH (rng.normal) et le Y de Sklearn partent du même 'bruit' initial.
Y_manual = tSNH(X_scaled, PERPLEXITY, T, ETA, ALPHA, D, random_state=RANDOM_STATE)
t1_manual = time()
print(f"-> tSNH terminé en {t1_manual - t0_manual:.4f} secondes.")


# --- 2. Exécution de l'implémentation Sklearn ---
print("\n[2/2] Exécution de l'Implémentation Sklearn (TSNE)...")
t0_sklearn = time()
tsne = TSNE(
n_components=D,
perplexity=PERPLEXITY,
max_iter=T,
random_state=RANDOM_STATE,
init='random',      # Force l'initialisation aléatoire
learning_rate=ETA,
method='exact',     # Force l'algorithme exact
n_jobs=1            # Pour un comportement cohérent
)
Y_sklearn = tsne.fit_transform(X_scaled)
t1_sklearn = time()
print(f"-> Sklearn TSNE terminé en {t1_sklearn - t0_sklearn:.4f} secondes.")
print(f"\nDivergence KL finale (accessible après l'entraînement): {tsne.kl_divergence_:.4f}")
```

```python
#c'est pour avoir la valeur de la divergence KL finale


# --- 3. Affichage des Résultats avec Matplotlib ---
fig, axes = plt.subplots(1, 2, figsize=(14, 6))
fig.suptitle(f'Comparaison t-SNE (Perplexity={PERPLEXITY}, N_iter={T})', fontsize=16)

# Graphique de notre implémentation
scatter_manual = axes[0].scatter(Y_manual[:, 0], Y_manual[:, 1], c=y, cmap=plt.cm.get_cmap("viridis", 3), s=40 )
axes[0].set_title(" tSNH")
axes[0].set_xlabel("Dimension 1")
axes[0].set_ylabel("Dimension 2")
axes[0].grid(True, linestyle='--', alpha=0.6)

legend1=axes[0].legend(*scatter_manual.legend_elements(), title="Classes", loc="lower left", labels=target_names)
axes[0].add_artist(legend1)
# Graphique de l'implémentation Sklearn
scatter_sklearn = axes[1].scatter(Y_sklearn[:, 0], Y_sklearn[:, 1], c=y, cmap=plt.cm.get_cmap("viridis", 3), s=40 )

axes[1].set_title("Sklearn TSNE (Exact, Random Init)")
axes[1].set_xlabel("Dimension 1")
axes[1].set_ylabel("Dimension 2")
axes[1].grid(True, linestyle='--', alpha=0.6)


legend2=axes[1].legend(*scatter_sklearn.legend_elements(), title="Classes", loc="lower left", labels=target_names)
axes[1].add_artist(legend2)
plt.tight_layout(rect=[0, 0.03, 1, 0.95]) # Ajuster l'espacement pour le titre
plt.show()

```

Discussion

On a eu comme résultat, la Figure (1).

On peut voir que les trois classes d'Iris (Setosa, Versicolor, Virginica) forme presque

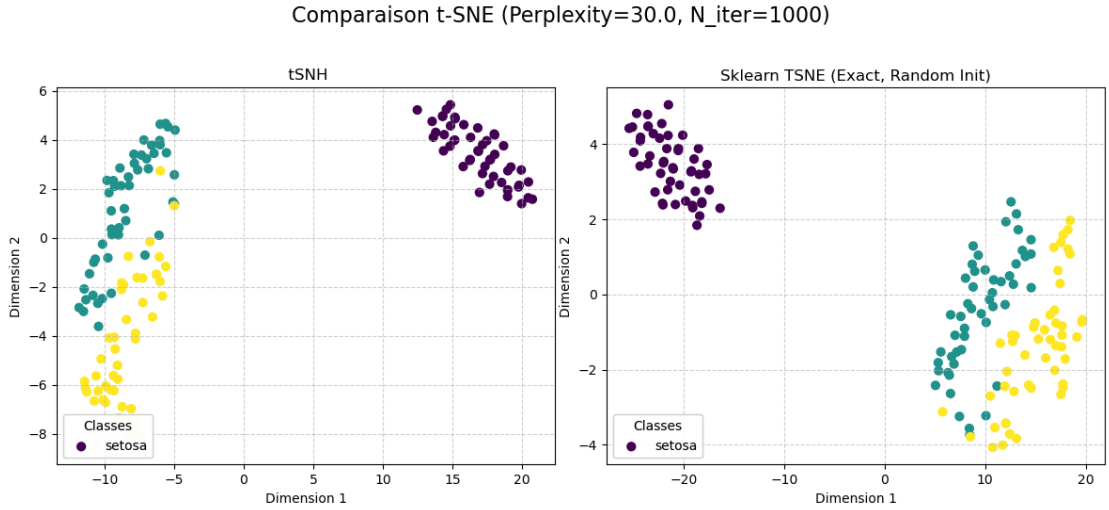

Figure 1: Comparaison de tSNH et TSNE de Sklearn sur Iris

des clusters distincts dans les deux visualisations.

En utilisant `np.random.seed(RANDOM_STATE)` avant d'appeler tSNH et en passant `random_state=RANDOM_STATE` à TSNE, on s'assure que les deux algorithmes commencent avec la même distribution initiale Y, ce qui est crucial pour une comparaison équitable.

L'implémentation Sklearn est plus lent (au alentour de 21.6200 secondes) que l'implémentation manuelle tSNH (au alentour de 15.6154 secondes), même si les deux utilisent la méthode exacte (qui calcule toutes les paires i,j)

Pour le jeu de données Iris (petit, n=150), l'écart de temps ne sera pas extrême mais mesurable. Cet écart augmenterait de façon exponentielle avec la taille du jeu de données n puisque la complexité de l'algorithme exact est en $O(n^2)$.

On constate aussi que le Divergence KL finale pour l'implementation sklearn est 0.1522 et celui de l'implémentation manuelle est 0,1574,il y a juste une difference de 0,0052.

Mais si on affiche la valeur de Y pour les 2 implementations et la valeur est differente pour les deux et c'est normal à cause de la non-convexité de Kullback-Leibler et les détails d'implémentation comme la gestion du momentum,le taux d'apprentissage et la precision des calculs.

6 Conclusion

On a présenté une nouvelle technique de visualisation de données de similarité qui est capable de retenir la structure locale des données tout en révélant également une structure globale importante (telle que les groupes ou clusters). La complexité computationnelle et la complexité de mémoire du t-SNE sont toutes deux en $O(n^2)$. Nos expériences sur une variété d'ensembles de données comme celle d'Iris qu'on a testé montrent que le t-SNE surpasse les techniques existantes de pointe (ou état de l'art) pour la visualisation d'une diversité de jeux de données réelles.